
Proyecto de desarrollo de aplicaciones web TriviUp

CICLO FORMATIVO DE GRADO SUPERIOR
Desarrollo de Aplicaciones Web (IFCS03)

Curso 2025-26

Autor/a/es:

Samuel Gómez Gutiérrez y Carlos Cortés Yagüe

Tutor/a:

José Luis González Sánchez

Departamento de Informática y Comunicaciones
I.E.S. Luis Vives

1 INTRODUCCIÓN

1.1 OBJETIVO

El objetivo principal de este proyecto es desarrollar una **plataforma de trivia multiplayer** en tiempo real que permita a los usuarios crear cuestionarios personalizados y competir con amigos en partidas sincronizadas.

Objetivos específicos

Implementar un sistema de juego en tiempo real mediante WebSockets

Proporcionar herramientas para la creación y gestión de cuestionarios

Garantizar autenticación segura con JWT y OAuth (Google)

Desplegar la aplicación en un entorno cloud (Railway)

Asegurar la calidad del código mediante tests automatizados

1.2 ALCANCE

Funcionalidades incluidas

Módulo	Funcionalidades
Autenticación	Registro, login, OAuth Google, gestión de sesión JWT
Cuestionarios	Crear, eliminar, listar, hacer públicos/privados
Preguntas	Añadir preguntas con 2-4 respuestas, marcar correcta, subir imágenes
Juego en tiempo real	Crear sala, unirse con código, turnos, puntuación, pause/resume
Perfil	Editar datos, subir foto de perfil, ver historial de partidas
Administración	Dashboard con estadísticas, gestión de usuarios (ban/activate)

Funcionalidades excluidas

- Chat en tiempo real entre jugadores (no requerido para trivia)
- Sistema de logros o badges
- Modo offline o sincronización offline
- Aplicación móvil nativa (solo web responsive)

Limitaciones técnicas

- Máximo 10 jugadores por sala
- Máximo 100 preguntas por cuestionario
- Imágenes de hasta 5MB por archivo
- Tiempo máximo de respuesta: 21 segundos

1.3 JUSTIFICACIÓN

Problema identificado

Las plataformas de trivia existentes o bien son estáticas (sin juego en tiempo real) o bien requieren software adicional para jugar en grupo. Existe necesidad de una solución web accesible que permita:

1. Crear contenido personalizado (propios cuestionarios)
2. Jugar de forma sincronizada con amigos
3. Acceder desde cualquier dispositivo sin instalar nada

Solución propuesta

TriviUp es una Single Page Application (SPA) que combina:

Frontend: Angular 21 con Signals para reactivity en tiempo real

Backend: .NET 9 con SignalR para comunicación bidireccional

Base de datos: PostgreSQL para persistencia relacional

Infraestructura: Railway para deployment cloud

Justificación de la arquitectura

La elección de una arquitectura full-stack con SignalR se justifica por:

Tiempo real: SignalR proporciona WebSockets manejados automáticamente con fallback a SSE/Long-polling

TypeScript en ambos extremos: Tipado compartido entre frontend y backend

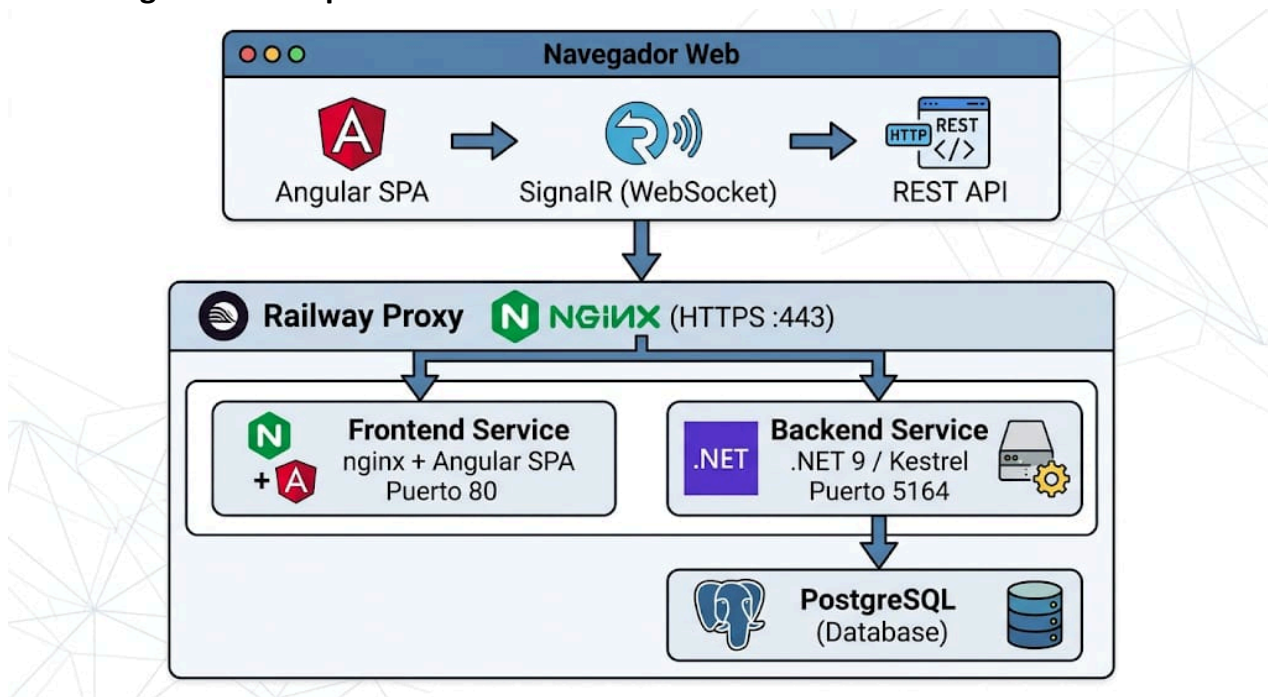
Single codebase: Más fácil de mantener que soluciones separadas

Escalabilidad: PostgreSQL + Railway permiten escalar verticalmente

2 IMPLEMENTACIÓN

2.1 ANÁLISIS DE LA APLICACIÓN

2.1.1. Diagrama de Arquitectura



2.1.2. Casos de Uso Principales

UC-01: Registrarse

Campo	Descripción
ID	UC-01
Nombre	Registrarse
Descripción	El usuario anónimo crea una nueva cuenta en la plataforma
Actor	Usuario Anónimo
Prioridad	Alta
Precondiciones	Usuario no tiene cuenta activa

Postcondiciones Usuario registrado y sesión iniciada

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Accede a la página de registro	Usuario Anónimo	
2	Muestra formulario con campos: nombre de usuario, email, contraseña		Sistema
3	Introduce datos: username, email, password	Usuario Anónimo	
4	Clic en "Crear Cuenta"	Usuario Anónimo	
5	Valida datos (formato email, longitud username, longitud password)		Sistema
6	Verifica que email y username no existen		Sistema
7	Hashea contraseña con bcrypt		Sistema
8	Crea usuario en base de datos		Sistema
9	Genera JWT token		Sistema
10	Guarda sesión en localStorage		Sistema
11	Redirige a página principal con sesión activa		Sistema

Flujos Alternativos:

Paso	Condición	Acción
5a	Datos inválidos	Mostrar mensaje de error específico, volver al paso 2
6a	Email ya existe	Mostrar error "El email ya está registrado", volver al paso 2
6b	Username ya existe	Mostrar error "El nombre de usuario ya está en uso", volver al paso 2

Excepciones:

Excepción	Acción del Sistema
Error de conexión a base de datos	Mostrar "Error del servidor. Inténtalo más tarde"

UC-02: Iniciar Sesión

Campo	Descripción
ID	UC-02
Nombre	Iniciar Sesión
Descripción	El usuario inicia sesión con email y contraseña
Actor	Usuario Anónimo, Usuario Registrado
Prioridad	Alta
Precondiciones	Usuario tiene cuenta creada
Postcondiciones	Sesión iniciada con JWT

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Accede a la página de login	Usuario	
2	Muestra formulario con campos: email, contraseña		Sistema
3	Introduce email y contraseña	Usuario	
4	Clic en "Iniciar Sesión"	Usuario	
5	Valida formato de email		Sistema
6	Busca usuario por email		Sistema
7	Verifica contraseña con bcrypt		Sistema
8	Verifica que usuario no está baneado		Sistema
9	Genera JWT token		Sistema
10	Actualiza lastLoginAt		Sistema

11	Guarda sesión en localStorage	Sistema
12	Redirige a página principal	Sistema

Flujos Alternativos:

Paso	Condición	Acción
6a	Email no encontrado	Mostrar error "Credenciales inválidas"
7a	Contraseña incorrecta	Mostrar error "Credenciales inválidas"
8a	Usuario baneado	Mostrar error "Tu cuenta ha sido suspendida"

UC-03: Cerrar Sesión

Campo	Descripción
ID	UC-03
Nombre	Cerrar Sesión
Descripción	El usuario cierra su sesión y vuelve a ser anónimo
Actor	Usuario Registrado
Prioridad	Alta
Precondiciones	Usuario tiene sesión activa
Postcondiciones	Sesión eliminada, usuario vuelve a ser anónimo

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Clic en avatar/nombre en navbar	Usuario	
2	Muestra dropdown con opciones		Sistema
3	Clic en "Cerrar Sesión"	Usuario	
4	Elimina token y usuario de localStorage		Sistema

5	Redirige a página principal	Sistema
6	Muestra navbar de usuario anónimo	Sistema

UC-04: Editar Perfil

Campo	Descripción
ID	UC-04
Nombre	Editar Perfil
Descripción	El usuario modifica sus datos personales
Actor	Usuario Registrado
Prioridad	Media
Precondiciones	Usuario autenticado
Postcondiciones	Datos de usuario actualizados

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Accede a "Editar Perfil" desde dropdown	Usuario	
2	Muestra formulario con datos actuales		Sistema
3	Modifica campos deseados (username, email, password)	Usuario	
4	Clic en "Guardar Cambios"	Usuario	
5	Valida nuevos datos		Sistema
6	Verifica unicidad de email/username		Sistema
7	Actualiza usuario en base de datos		Sistema
8	Muestra mensaje de éxito		Sistema

9	Redirige a perfil	Sistema
---	-------------------	---------

Flujos Alternativos:

Paso	Condición	Acción
5a	Datos inválidos	Mostrar errores de validación inline
6a	Email/Username duplicado	Mostrar error específico

UC-05: Login con Google

Campo	Descripción
ID	UC-05
Nombre	Login con Google
Descripción	El usuario inicia sesión usando su cuenta de Google
Actor	Usuario Anónimo
Prioridad	Media
Precondiciones	Usuario tiene cuenta de Google
Postcondiciones	Usuario registrado (si es nuevo) y sesión iniciada

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Clic en "Continuar con Google"	Usuario	
2	Redirige a página de Google OAuth		Sistema
3	Usuario autoriza acceso a su cuenta Google	Usuario	
4	Google redirige a /auth/google/callback con código		Sistema

5	Intercambia código por access_token de Google	Sistema
6	Obtiene datos del usuario de Google	Sistema
7	Busca usuario por google_id	Sistema
8	Si no existe, crea nuevo usuario	Sistema
9	Genera JWT token	Sistema
10	Redirige a /auth/callback con token y user en URL	Sistema
11	Guarda sesión en localStorage	Sistema
12	Redirige a página principal	Sistema

Flujos Alternativos:

Paso	Condición	Acción
3a	Usuario rechaza autorización	Redirige a página de login con error

UC-26: Subir Foto de Perfil

Campo	Descripción
ID	UC-26
Nombre	Subir Foto de Perfil
Descripción	El usuario sube una imagen como foto de perfil
Actor	Usuario Registrado
Prioridad	Baja
Precondiciones	Usuario autenticado
Postcondiciones	Foto de perfil actualizada

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Accede a "Editar Perfil"	Usuario	

2	Clic en "Cambiar Foto"	Usuario
3	Abre selector de archivos del sistema	Sistema
4	Selecciona imagen (JPG, PNG, GIF, WebP)	Usuario
5	Valida tipo de archivo	Sistema
6	Valida tamaño (máx 5MB)	Sistema
7	Muestra preview de la imagen	Sistema
8	Clic en "Guardar Cambios"	Usuario
9	Sube imagen al servidor	Sistema
10	Guarda URL en base de datos	Sistema
11	Muestra mensaje de éxito	Sistema

Flujos Alternativos:

Paso	Condición	Acción
5a	Formato no soportado	Mostrar error "Formato de imagen no válido"
6a	Archivo > 5MB	Mostrar error "La imagen debe ser menor a 5MB"

UC-06: Explorar Cuestionarios Públicos

Campo	Descripción
ID	UC-06
Nombre	Explorar Cuestionarios Públicos
Descripción	El usuario navega y busca cuestionarios públicos
Actor	Usuario Anónimo, Usuario Registrado
Prioridad	Alta

Precondiciones Ninguna

Postcondiciones Ninguna

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Accede a "/quizzes/public" o clic en "Explorar"	Usuario	
2	Muestra lista de cuestionarios públicos (página 1)		Sistema
3	Opcional: escribe en buscador	Usuario	
4	Clic en "Buscar" o presiona Enter	Usuario	
5	Filtra por nombre y muestra resultados		Sistema
6	Opcional: clic en paginación	Usuario	
7	Muestra página seleccionada		Sistema

Flujos Alternativos:

Paso	Condición	Acción
5a	Sin resultados	Mostrar mensaje "No se encontraron cuestionarios"

UC-07: Ver Detalle de Cuestionario

Campo	Descripción
ID	UC-07
Nombre	Ver Detalle de Cuestionario
Descripción	El usuario ve los detalles completos de un cuestionario

Actor	Usuario Anónimo, Usuario Registrado
Prioridad	Alta
Precondiciones	Ninguna
Postcondiciones	Contador de visitas incrementado

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Clic en un cuestionario de la lista	Usuario	
2	Registra visita (POST /api/quizzes/{id}/visit)		Sistema
3	Obtiene cuestionario con preguntas		Sistema
4	Muestra página de detalle con:		Sistema
	- Título y creador		
	- Número de preguntas		
	- Likes y visitas		
	- Botón "Crear Sala" (si logueado)		
	- Botón "Me gusta"		
	- Lista de preguntas (sin revelar correctas)		

UC-08: Crear Cuestionario

Campo	Descripción
ID	UC-08
Nombre	Crear Cuestionario
Descripción	El usuario crea un nuevo cuestionario con preguntas
Actor	Usuario Registrado

Prioridad Alta

Precondiciones Usuario autenticado

Postcondiciones Cuestionario creado con código único

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Clic en "Crear Cuestionario" en navbar	Usuario	
2	Muestra formulario vacío		Sistema
3	Introduce nombre del cuestionario	Usuario	
4	Marca "Cuestionario público" si lo desea	Usuario	
5	Clic en "Añadir Pregunta"	Usuario	
6	Muestra formulario de pregunta		Sistema
7	Introduce enunciado	Usuario	
8	Opcional: sube imagen	Usuario	
9	Añade respuestas (mínimo 2, máximo 4)	Usuario	
10	Marca una respuesta como correcta	Usuario	
11	Clic en "Guardar Pregunta"	Usuario	
12	Pregunta aparece en la lista		Sistema
13	Repite pasos 5-12 para más preguntas		
14	Clic en "Guardar Cuestionario"	Usuario	

15	Valida: nombre no vacío, al menos 1 pregunta	Sistema
16	Genera código único de 6 caracteres	Sistema
17	Guarda cuestionario en base de datos	Sistema
18	Muestra cuestionario creado con código QR	Sistema

Flujos Alternativos:

Paso	Condición	Acción
11a	Menos de 2 respuestas	Mostrar error "Mínimo 2 respuestas"
11b	Ninguna respuesta marcada como correcta	Mostrar error "Marca una respuesta como correcta"
15a	Nombre vacío	Mostrar error junto al campo
15b	Sin preguntas	Mostrar error "Añade al menos una pregunta"

UC-09: Editar Cuestionario

Campo	Descripción
ID	UC-09
Nombre	Editar Cuestionario
Descripción	El propietario modifica su cuestionario
Actor	Usuario Registrado (propietario)
Prioridad	Alta
Precondiciones	Usuario es propietario del cuestionario
Postcondiciones	Cuestionario actualizado

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Accede a "Mis Cuestionarios"	Usuario	

2	Muestra lista de cuestionarios del usuario	Sistema
3	Clic en botón "Editar" del cuestionario	Usuario
4	Verifica que no está en uso en sala activa	Sistema
5	Muestra formulario con datos actuales	Sistema
6	Modifica datos y/o preguntas	Usuario
7	Clic en "Guardar Cambios"	Usuario
8	Valida datos	Sistema
9	Actualiza en base de datos	Sistema
10	Muestra mensaje de éxito	Sistema

Flujos Alternativos:

Paso	Condición	Acción
4a	Cuestionario en uso	Mostrar error "No puedes editar mientras hay una sala activa"

UC-10: Eliminar Cuestionario

Campo	Descripción
ID	UC-10
Nombre	Eliminar Cuestionario
Descripción	El propietario elimina su cuestionario
Actor	Usuario Registrado (propietario)
Prioridad	Alta
Precondiciones	Usuario es propietario del cuestionario
Postcondiciones	Cuestionario eliminado (cascade delete)

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Accede a "Mis Cuestionarios"	Usuario	
2	Clic en botón "Eliminar" del cuestionario	Usuario	
3	Muestra modal de confirmación		Sistema
4	Confirma eliminación	Usuario	
5	Elimina preguntas y respuestas en cascada		Sistema
6	Elimina cuestionario		Sistema
7	Muestra mensaje de éxito		Sistema
8	Actualiza lista de cuestionarios		Sistema

UC-11: Dar Like a Cuestionario

Campo	Descripción
ID	UC-11
Nombre	Dar Like a Cuestionario
Descripción	El usuario vota positivamente un cuestionario público
Actor	Usuario Anónimo, Usuario Registrado
Prioridad	Media
Precondiciones	Ninguna
Postcondiciones	Contador de likes incrementado

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Estando en detalle de cuestionario	Usuario	

2	Clic en icono de corazón vacío	Usuario
3	Llama POST /api/quizzes/{id}/like	Sistema
4	Incrementa contador de likes	Sistema
5	Actualiza icono a corazón lleno	Sistema
6	Actualiza número de likes	Sistema

UC-12: Quitar Like a Cuestionario

Campo	Descripción
ID	UC-12
Nombre	Quitar Like a Cuestionario
Descripción	El usuario elimina su voto de un cuestionario
Actor	Usuario Anónimo, Usuario Registrado
Prioridad	Media
Precondiciones	Usuario ya dio like
Postcondiciones	Contador de likes decrementado

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Estando en detalle de cuestionario (con like activo)	Usuario	
2	Clic en icono de	Usuario	
3	Llama DELETE /api/quizzes/{id}/like		Sistema
4	Decrementa contador de likes		Sistema
5	Actualiza icono a corazón vacío		Sistema

6 Actualiza número de likes Sistema

UC-13: Crear Sala de Juego

Campo	Descripción
ID	UC-13
Nombre	Crear Sala de Juego
Descripción	El usuario crea una sala para jugar un cuestionario
Actor	Usuario Registrado
Prioridad	Alta
Precondiciones	Usuario autenticado, cuestionario seleccionado
Postcondiciones	Sala creada con código único

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Estando en detalle de cuestionario	Usuario	
2	Clic en "Crear Sala"	Usuario	
3	Conecta a SignalR Hub		Sistema
4	Invoca método CreateGame en el Hub		Sistema
5	Crea GameRoom en memoria		Sistema
6	Genera código de 6 caracteres		Sistema

7	Envía código al cliente	Sistema
8	Muestra sala de espera con:	Sistema
	- Código de sala	
	- Código QR	
	- Lista de jugadores (vacía)	
	- Botón "Iniciar Juego" (deshabilitado)	

UC-14: Unirse a Sala

Campo	Descripción
ID	UC-14
Nombre	Unirse a Sala
Descripción	El usuario se une a una sala existente
Actor	Usuario Anónimo, Usuario Registrado
Prioridad	Alta
Precondiciones	Código de sala válido
Postcondiciones	Usuario añadido a la sala

Flujo Principal (Usuario Registrado):

Paso	Acción	Actor	Sistema
1	Accede a página "Unirse"	Usuario	
2	Introduce código de sala	Usuario	
3	Clic en "Entrar"	Usuario	
4	Conecta a SignalR con JWT		Sistema

5	Invoca JoinGame con userId y username	Sistema
6	Valida código de sala	Sistema
7	Valida que usuario no está baneado	Sistema
8	Añade jugador a la sala	Sistema
9	Notifica a todos los jugadores (PlayerJoined)	Sistema
10	Muestra sala de espera	Sistema

Flujo Alternativo (Usuario Anónimo):

Paso	Acción	Actor	Sistema
1	Accede a página "Unirse"	Usuario	
2	Introduce código de sala y nombre de usuario	Usuario	
3	Clic en "Entrar como Invitado"	Usuario	
4	Conecta a SignalR sin JWT		Sistema
5	Invoca JoinGame solo con username		Sistema
6	Continúa desde paso 6...		

Flujos Alternativos:

Paso	Condición	Acción
6a	Código no existe	Mostrar error "Sala no encontrada"
7a	Usuario baneado	Mostrar error "No puedes unirte"
7b	Sala llena (10 jugadores)	Mostrar error "La sala está llena"

UC-15: Iniciar Juego

Campo	Descripción
ID	UC-15

Nombre	Iniciar Juego
Descripción	El owner inicia la partida con los jugadores presentes
Actor	Usuario Registrado (owner de la sala)
Prioridad	Alta
Precondiciones	Usuario es owner, mínimo 2 jugadores
Postcondiciones	Juego started, countdown iniciado

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Estando en sala de espera	Owner	
2	Verifica que hay mínimo 2 jugadores		Sistema
3	Clic en "Iniciar Juego"	Owner	
4	Habilita botón (mínimo 2 jugadores)		Sistema
5	Invoca StartGame en SignalR	Owner	
6	Baraja orden de jugadores		Sistema
7	Baraja preguntas del cuestionario		Sistema
8	Cambia estado a "Starting"		Sistema
9	Envía countdown (3 segundos) a todos		Sistema
10	Cambio de estado a "Playing"		Sistema
11	Envía primera pregunta al primer jugador		Sistema
12	Inicia timer de 21 segundos		Sistema

Flujos Alternativos:

Paso	Condición	Acción
------	-----------	--------

2a Menos de 2 jugadores Botón deshabilitado, tooltip "Necesitas al menos 2 jugadores"

UC-16: Responder Pregunta

Campo	Descripción
ID	UC-16
Nombre	Responder Pregunta
Descripción	El jugador actual responde la pregunta en su turno
Actor	Usuario Registrado, Usuario Anónimo
Prioridad	Alta
Precondiciones	Es el turno del jugador
Postcondiciones	Respuesta registrada, puntos calculados

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Jugador recibe TurnStarted con pregunta		Sistema
2	Muestra pregunta y 4 opciones (formas)		Sistema
3	Inicia countdown de 21 segundos		Sistema
4	Jugador selecciona una opción	Usuario	

5	Detiene timer	Sistema
6	Envía SubmitAnswer al servidor	Sistema
7	Valida que es el turno del jugador	Sistema
8	Compara respuesta con la correcta	Sistema
9	Calcula puntos: $100 + (\text{tiempo_restante} \times 10)$	Sistema
10	Envía TurnResult a todos los jugadores	Sistema
11	Pasa turno al siguiente jugador	Sistema
12	Si quedan preguntas, envía siguiente pregunta	Sistema
13	Si no quedan, cambia estado a "Finished"	Sistema

Flujo Alternativo (Timeout):

Paso	Acción	Actor	Sistema
3	Timer llega a 0		Sistema
4	Considered como incorrecta		Sistema
5	0 puntos		Sistema
6	Continúa flujo desde paso 10		Sistema

UC-17: Pausar Juego

Campo	Descripción
ID	UC-17
Nombre	Pausar Juego
Descripción	El owner pausa la partida
Actor	Usuario Registrado (owner)
Prioridad	Media
Precondiciones	Usuario es owner, juego en progreso

Postcondiciones Juego pausado, timer detenido

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Owner clic en "Pausar"	Owner	
2	Invoca PauseGame en SignalR	Owner	
3	Detiene timer actual		Sistema
4	Guarda tiempo restante		Sistema
5	Cambia estado a "Paused"		Sistema
6	Envía GamePaused a todos		Sistema
7	Muestra mensaje "Juego pausado" a todos		Sistema

UC-18: Reanudar Juego

Campo	Descripción
ID	UC-18
Nombre	Reanudar Juego
Descripción	El owner reanuda la partida pausada
Actor	Usuario Registrado (owner)
Prioridad	Media
Precondiciones	Usuario es owner, juego pausado
Postcondiciones	Juego en progreso

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Owner clic en "Reanudar"	Owner	

2	Invoca ResumeGame en SignalR	Owner
3	Restaura timer con tiempo restante	Sistema
4	Cambia estado a "Playing"	Sistema
5	Envía GameResumed a todos	Sistema
6	Continúa el turno actual	Sistema

UC-19: Expulsar Jugador

Campo	Descripción
ID	UC-19
Nombre	Expulsar Jugador
Descripción	El owner expulsa a un jugador de la sala
Actor	Usuario Registrado (owner)
Prioridad	Baja
Precondiciones	Usuario es owner, no es el mismo jugador
Postcondiciones	Jugador desconectado de la sala

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Owner clic en "Expulsar" junto al jugador	Owner	
2	Invoca KickPlayer en SignalR	Owner	
3	Envía PlayerKicked al jugador expulsado		Sistema
4	Desconecta al jugador de la sala		Sistema

5	Notifica a los demás jugadores	Sistema
6	Jugador expulsado ve mensaje y es redirigido	Sistema

UC-20: Ver Resultados Finales

Campo	Descripción
ID	UC-20
Nombre	Ver Resultados Finales
Descripción	Se muestra la clasificación final de la partida
Actor	Usuario Registrado, Usuario Anónimo
Prioridad	Alta
Precondiciones	Juego ha terminado
Postcondiciones	Partida guardada en historial

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Todos los jugadores han respondido la última pregunta		Sistema
2	Cambia estado a "Finished"		Sistema
3	Calcula clasificación final		Sistema
4	Para cada jugador, guarda en GameHistory		Sistema

5	Envía GameFinished a todos los jugadores	Sistema
6	Muestra pantalla de resultados:	Sistema
	- Clasificación ordenada por puntos	
	- Puntos totales de cada jugador	
	- Respuestas correctas/incorrectas	
7	Opción de "Volver al inicio" o "Jugar de nuevo"	Sistema

UC-21: Ver Historial de Partidas

Campo	Descripción
ID	UC-21
Nombre	Ver Historial de Partidas
Descripción	El usuario ve sus partidas pasadas
Actor	Usuario Registrado
Prioridad	Baja
Precondiciones	Usuario autenticado
Postcondiciones	Ninguna

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Accede a "Historial" desde navbar	Usuario	
2	Llama GET /api/game/history		Sistema
3	Obtiene lista de partidas del usuario		Sistema

4	Muestra lista ordenada por fecha (más recientes primero)	Sistema
5	Cada elemento muestra: quiz, fecha, posición	Sistema
6	Opcional: clic en una partida	Usuario
7	Muestra detalle de la partida	Sistema

UC-22: Ver Dashboard de Administración

Campo	Descripción
ID	UC-22
Nombre	Ver Dashboard de Administración
Descripción	El administrador ve estadísticas del sistema
Actor	Administrador
Prioridad	Media
Precondiciones	Usuario con rol ADMIN
Postcondiciones	Ninguna

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Admin accede a "/admin"	Administrador	
2	Verifica rol ADMIN		Sistema
3	Llama GET /api/admin/stats		Sistema

4	Obtiene estadísticas:	Sistema
	- Total usuarios	
	- Total cuestionarios	
	- Total partidas jugadas	
	- Usuarios activos últimas 24h	
	- Cuestionario más visitado	
	- Cuestionario con más likes	
5	Muestra dashboard con datos y gráficos	Sistema

UC-23: Gestionar Usuarios

Campo	Descripción
ID	UC-23
Nombre	Gestionar Usuarios
Descripción	El administrador lista y gestiona usuarios del sistema
Actor	Administrador
Prioridad	Media
Precondiciones	Usuario con rol ADMIN
Postcondiciones	Datos de usuario modificados

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Admin accede a "/admin/users"	Administrador	
2	Verifica rol ADMIN		Sistema
3	Llama GET /api/users		Sistema

4	Obtiene lista de todos los usuarios	Sistema
5	Muestra tabla con: nombre, email, rol, estado, fecha registro	Sistema
6	Admin puede: buscar, filtrar, paginar	Administrador
7	Admin selecciona un usuario	Administrador
8	Muestra opciones: editar, banear, activar	Sistema

UC-24: Banear Usuario

Campo	Descripción
ID	UC-24
Nombre	Banear Usuario
Descripción	El administrador suspende a un usuario
Actor	Administrador
Prioridad	Media
Precondiciones	Usuario con rol ADMIN, usuario a banear no es admin
Postcondiciones	Usuario marcado como baneado

Flujo Principal:

Paso	Acción	Actor	Sistema
1	Admin está en gestión de usuarios	Administrador	
2	Clic en "Banear" junto al usuario	Administrador	
3	Muestra modal de confirmación		Sistema

4	Admin confirma	Administrador
5	Llama POST /api/users/{id}/ban	Sistema
6	Actualiza is_banned = true	Sistema
7	Muestra mensaje de éxito	Sistema
8	Actualiza tabla (usuario aparece como baneado)	Sistema

Flujos Alternativos:

Paso	Condición	Acción
2a	Usuario es ADMIN	Botón deshabilitado

UC-25: Activar Usuario

Campo	Descripción
ID	UC-25
Nombre	Activar Usuario
Descripción	El administrador restaura a un usuario baneado
Actor	Administrador
Prioridad	Media
Precondiciones	Usuario con rol ADMIN, usuario seleccionado está baneado
Postcondiciones	Usuario marcado como activo

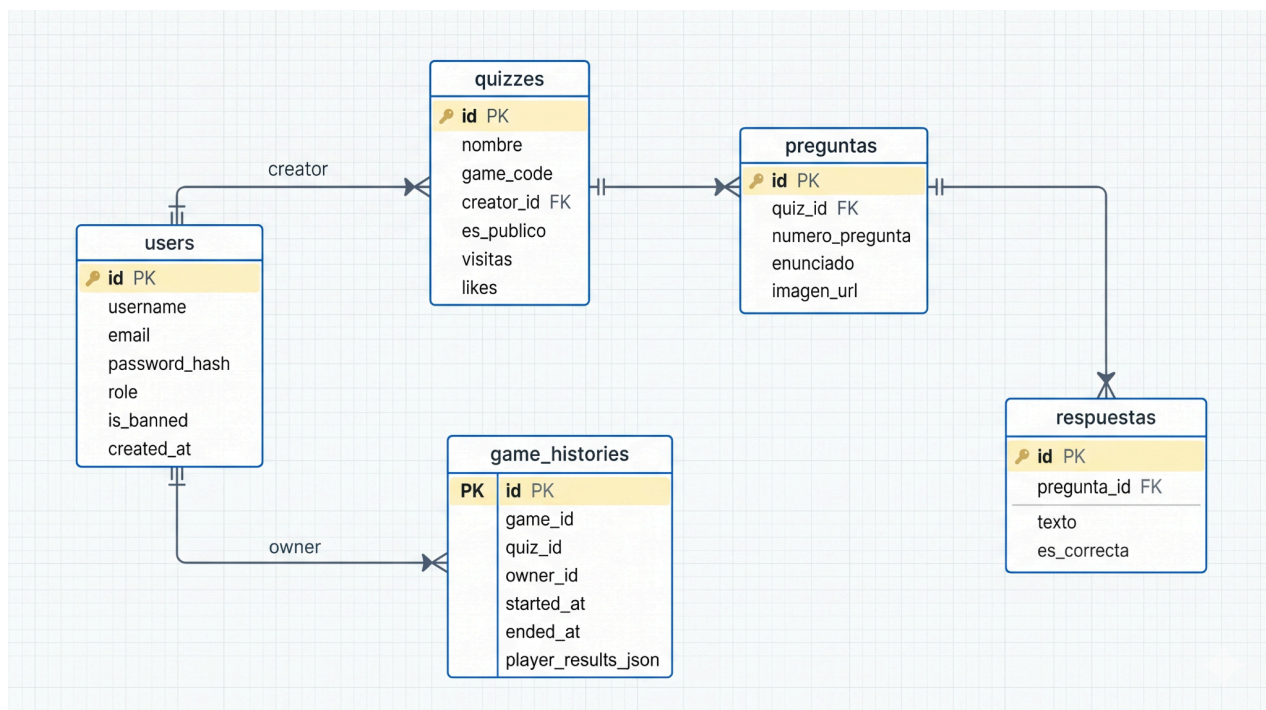
Flujo Principal:

Paso	Acción	Actor	Sistema
1	Admin está en gestión de usuarios	Administrador	
2	Clic en "Activar" junto al usuario baneado	Administrador	
3	Llama POST /api/users/{id}/activate		Sistema

4	Actualiza is_banned = false	Sistema
5	Muestra mensaje de éxito	Sistema
6	Actualiza tabla (usuario aparece como activo)	Sistema

2.2 DISEÑO

2.2.1. Diagrama Entidad-Relación



2.2.2. Estructura del Proyecto

Backend (.NET 9)

```
TriviUpBackend/
├── TriviUpBackend/
│   ├── Controllers/           # API Controllers
│   ├── Services/              # Lógica de negocio
│   │   ├── Auth/              # JWT, Google OAuth
│   │   ├── Cache/             # MemoryCache
│   │   └── Game/              # SignalR Hub, TurnManager
│   ├── Repositories/          # Acceso a datos (EF Core)
│   ├── Models/                # Entidades de dominio
│   ├── Entities/              # Entidades de base de datos
│   ├── DTO/                   # Data Transfer Objects
│   ├── Errors/                # Tipos de errores
│   ├── Common/                # Código compartido
│   │   └── Storage/           # Servicio de archivos
│   ├── Infrastructure/         # DI, config, middleware
│   ├── Database/              # DbContext
│   ├── Data/                  # Interceptors
│   ├── Game/                  # Lógica de juego
│   │   ├── DTOs/              # Game data transfer objects
│   │   ├── Hubs/              # SignalR Hub
│   │   ├── Services/          # GameService, TurnManager
│   │   └── Configuration/     # GameOptions
│   └── Program.cs              # Entry point
└── TriviUpTest/               # Tests unitarios (xUnit + Moq)
```

Frontend (Angular 21)

```
TriviUpFrontend/
├── src/app/
│   ├── app.ts                 # Componente raíz
│   ├── app.routes.ts          # Rutas de la aplicación
│   ├── app.config.ts          # Configuración (providers)
│   ├── auth/                  # Módulo de autenticación
│   ├── home/                  # Página de inicio
│   ├── cuestionarios/         # Módulo de cuestionarios
│   │   ├── components/        # QuizList, QuizDetail, QuizForm, etc.
│   │   ├── models/            # Interfaces TypeScript
│   │   ├── pages/             # CrearCuestionario
│   │   └── services/           # CuestionarioService, QuizzesPublicService
│   ├── game/                  # Módulo de juego
│   │   ├── components/        # QRCode, GameLobby
│   │   ├── models/
│   │   ├── pages/             # GameRoom, GameHistory, JoinRoom
│   │   └── services/           # GameSignalRService
│   ├── user/                  # Módulo de usuario
│   ├── admin/                 # Módulo de administración
│   ├── shared/                # Componentes compartidos
│   │   ├── components/        # Navbar, ProfilePhoto, AnswerShape
│   │   ├── pages/             # Informacion, Contacto
│   │   └── services/           # AudioService
│   └── services/              # UserService, ImageService
├── nginx.conf                 # Configuración nginx
└── package.json
```

2.2.3. Rutas de la Aplicación

Ruta	Componente	Auth	Descripción
------	------------	------	-------------

/	Home	-	Página de inicio
/auth	Auth	-	Login/Registro
/auth/callback	AuthCallback	-	Callback Google OAuth
/perfil	Profile	-	Ver perfil
/perfil/editar	ProfileEdit	-	Editar perfil
/cuestionarios/crear	CrearCuestionarioPage	JWT	Crear cuestionario
/cuestionarios/mis-cuestionarios	QuizListComponent	JWT	Mis cuestionarios
/quizzes/public	QuizzesPublicComponent	-	Cuestionarios públicos
/cuestionarios/:id	QuizDetailComponent	-	Detalle de cuestionario
/game/history	GameHistoryComponent	JWT	Historial de partidas
/game/history/:gameId	GameDetailsComponent	JWT	Detalle de partida
/unirse	JoinRoomComponent	-	Unirse por código
/game/:roomCode	GameRoomComponent	-	Sala de juego activa
/admin	AdminDashboardComponent	ADMIN	Dashboard admin
/admin/users	AdminUsersComponent	ADMIN	Gestión de usuarios
/informacion	InformacionComponent	-	Página de información
/contacto	ContactoComponent	-	Página de contacto

2.3 IMPLEMENTACIÓN

2.3.1. Backend - Controladores y APIs

AuthController (/auth)

Método	Ruta	Auth	Descripción
POST	/auth/signup	No	Registro de usuario
POST	/auth/signin	No	Inicio de sesión
GET	/auth/google	No	Inicia flujo Google OAuth
GET	/auth/google/callback	No	Callback de Google OAuth

UsersController (/users)

Método	Ruta	Auth	Descripción
GET	/users	ADMIN	Listar todos los usuarios
GET	/users/{id}	ADMIN	Obtener usuario por ID
PUT	/users/{id}	ADMIN	Actualizar usuario
POST	/users/{id}/ban	ADMIN	Banear usuario
POST	/users/{id}/activate	ADMIN	Activar usuario baneado
PUT	/users/profile-photo	JWT	Actualizar foto de perfil
DELETE	/users/profile-photo	JWT	Eliminar foto de perfil
GET	/users/me	JWT	Obtener usuario actual
PUT	/users/me	JWT	Actualizar perfil propio

CuestionariosController (/api/cuestionarios)

Método	Ruta	Auth	Descripción
POST	/api/cuestionarios	JWT	Crear cuestionario

GET	/api/cuestionarios	JWT	Listar cuestionarios (paginado)
GET	/api/cuestionarios/{id}	JWT	Obtener cuestionario
GET	/api/cuestionarios/gamecode/{gameCode}	JWT	Obtener por gameCode
GET	/api/cuestionarios/mis-cuestionarios	JWT	Cuestionarios del usuario
PUT	/api/cuestionarios/{id}	JWT (owner)	Actualizar cuestionario
DELETE	/api/cuestionarios/{id}	JWT (owner)	Eliminar cuestionario

QuizzesController (/api/quizzes)

Método	Ruta	Auth	Descripción
GET	/api/quizzes/public	No	Listar cuestionarios públicos
POST	/api/quizzes/{id}/like	No	Dar like a quiz
DELETE	/api/quizzes/{id}/like	No	Quitar like
POST	/api/quizzes/{id}/visit	No	Registrar visita

GamesController (/api/game)

Método	Ruta	Auth	Descripción
GET	/api/game/history	JWT	Historial de partidas del usuario
GET	/api/game/{gameId}	JWT	Detalles de una partida

AdminController (/api/admin)

Método	Ruta	Auth	Descripción
GET	/api/admin/stats	ADMIN	Estadísticas generales

2.3.2. Backend - SignalR Hub (/hubs/game)

Métodos del Cliente (el servidor llama a estos):

Método	Datos	Descripción
GameCreated	GameLobbyState	Sala creada
PlayersList	Player[]	Lista de jugadores
PlayerJoined	Player	Nuevo jugador
PlayerKicked	number (userId)	Jugador expulsado
GameStarted	GameStateDto	Juego iniciado
TurnStarted	TurnStartedDto	Nuevo turno
TurnResult	TurnResult	Resultado del turno
TurnTimeout	{playerId}	Tiempo agotado
GamePaused	{roomCode}	Juego pausado
GameResumed	{roomCode, timeRemaining}	Juego reanudado
GameFinished	GameResult	Juego terminado

Métodos del Servidor (el cliente llama a estos):

Método	Parámetros	Auth	Descripción
CreateGame	quizId	JWT	Crear sala
JoinGame	roomCode, userId?, username?	JWT/Anon	Unirse a sala
LeaveGame	roomCode	-	Salir de sala
StartGame	roomCode	Owner	Iniciar juego
SubmitAnswer	roomCode, questionId, answerIndex, timeRemaining	-	Enviar respuesta
PauseGame	roomCode	Owner	Pausar juego

ResumeGame	roomCode	Owner	Reanudar juego
KickPlayer	roomCode, playerIdToKick	Owner	Expulsar jugador

2.3.3. Frontend - Servicios Principales

AuthService

```

signIn(payload: SignInPayload): Observable<AuthResponse>
signUp(payload: SignUpPayload): Observable<AuthResponse>
saveSession(response: AuthResponse): void
getToken(): string | null
getUser(): AuthUser | null
isLoggedIn(): boolean
logout(): void
getUserSignal(): Signal<AuthUser | null>

```

GameSignalrService

```

connect(token?: string): Promise<void>
connectAnonymously(userId: number, username: string): Promise<void>
createGame(quizId: number): Promise<string>
joinGame(roomCode: string, userId?: number, username?: string): Promise<void>
startGame(roomCode: string): Promise<void>
submitAnswer(roomCode: string, questionId: number, answerIndex: number, timeRemaining: number):
Promise<void>
pauseGame(roomCode: string): Promise<void>

```

```
resumeGame(roomCode: string): Promise<void>
```

```
kickPlayer(roomCode: string, playerId: number): Promise<void>
```

2.3.4. Sistema de Puntuación

Puntos = 100 + (tiempo_restante × 10)

Máximo bonus de tiempo = 200

Total máximo por pregunta = 300 puntos

2.3.5. Estados del Juego

Waiting → Sala creada, esperando jugadores

Starting → Countdown antes de empezar (3 segundos)

Playing → Juego en progreso

Paused → Pausado por el owner

Finished → Juego terminado

2.3.6. Autenticación JWT

Claims del token:

```
new Claim(JwtRegisteredClaimNames.Sub, user.Id.ToString())
```

```
new Claim(JwtRegisteredClaimNames.Email, user.Email)
```

```
new Claim(ClaimTypes.Role, user.Role)
```

```
new Claim("username", user.Username)
```

```
new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
```

Configuración:

- **Key:** Jwt:Key (32+ caracteres)
- **Issuer:** TriviUpBackend
- **Audience:** TriviUpApp
- **Expiración:** 60 minutos

2.3.7. Base de Datos - Primary Keys

- **users:** Unique en username, email, google_id
- **quizzes:** Unique en game_code
- **game_histories:** Unique en game_id

2.4 IMPLANTACIÓN

El proyecto usa Railway para deployment.

Servicios:

- **TriviUp-Backend:** API .NET 9 en puerto 5164
- **TriviUp-Frontend:** nginx + Angular SPA en puerto 80
- **Postgres:** Base de datos PostgreSQL

URLs de Producción:

Servicio	URL
Frontend	https://triviup.up.railway.app
Backend API	https://triviup-backend.up.railway.app

2.5 DOCUMENTACIÓN

2.5.1. Decisiones Técnicas - Backend

.NET 9 / ASP.NET Core Web API

Elección: .NET 9 con ASP.NET Core Web API como framework principal.

Alternativa	Ventajas de .NET 9
Node.js / Express	Rendimiento superior en operaciones CPU-bound; tipado estático con C#; LINQ queries typed
Python / Django/FastAPI	5-10x más rápido en benchmarks; tipado estático; async/await nativo
Java / Spring Boot	Sintaxis más concisa con C# record types; configuración más simple

Trade-offs:

- Curva de aprendizaje mayor que Node.js para developers Junior
- Ecosistema npm más grande para ciertas herramientas

Caso de uso en TriviUp:

- Middleware pipeline para auth, logging, CORS
- Kestrel como servidor HTTP de alto rendimiento
- Integración nativa con SignalR

Entity Framework Core

Elección: Entity Framework Core como ORM principal con PostgreSQL.

Alternativa	Ventajas de EF Core
ADO.NET puro	Productividad 5-10x mayor en CRUD; migrations automáticas versionables
Dapper	EF Core ofrece más abstracción; cambio de InMemory a PostgreSQL sin cambios

Trade-offs:

- Para queries muy complejas, EF Core tiene overhead vs SQL puro
- Learning curve para evitar N+1 queries

PostgreSQL

Elección: PostgreSQL como base de datos relacional.

Alternativa	Ventajas de PostgreSQL
MySQL / MariaDB	Tipos más ricos (JSONB, arrays); full-text search nativo
MongoDB	Consistencia fuerte ACID; joins entre colecciones
SQL Server	Cross-platform; sin costo de licencia

SignalR (WebSocket)

Elección: SignalR para comunicación en tiempo real.

Alternativa	Ventajas de SignalR
Socket.io	Integración primera con .NET; automatic reconnection
Pure WebSockets	Fallback a SSE o Long Polling; automatic heartbeats

Caso de uso: Notificaciones instantáneas, estado de sala compartido, reconnection handling.

JWT Bearer Authentication

Elección: JWT Bearer tokens para autenticación stateless.

- JWT es stateless (no storage server-side)
- Escalado horizontal sin session store
- Compatible con mobile/SPA

Clean Architecture

Elección: Clean Architecture con Controllers → Services → Repositories.

Controllers/ → HTTP layer
Services/ → Business logic
Repositories/ → Data access (EF Core)
DTOs/ → API contracts
Entities/ → Database entities
Errors/ → Domain errors (Result types)

2.5.2. Decisiones Técnicas - Frontend

Angular 21

Elección: Angular 21 con TypeScript obligatorio.

Alternativa	Ventajas de Angular
React	Ecosistema enorme
Vue 3	Curva suave
Svelte	Bundle pequeño

Ventajas aplicadas:

- TypeScript nativo y obligatorio (`strict: true`, `strictTemplates: true`)
- RxJS para flujos asíncronos
- Inyección de Dependencias (DI)
- Standalone Components (desde Angular 14, maduro en v21)
- Signals para estado reactivo

Standalone Components

Elección: Componentes independientes sin NgModules.

- Menos boilerplate: no hay archivos `.module.ts`
- Importación directa: cada componente importa lo que necesita
- Lazy loading simplificado con `loadComponent()`
- Mejor tree-shaking

TailwindCSS v4 + DaisyUI 5

Elección: TailwindCSS v4 con DaisyUI 5 para componentes.

Librería	Framework	Tamaño bundle
Angular Material	Angular	~300KB
DaisyUI	TailwindCSS	~20KB

Ventajas:

- Integración nativa con TailwindCSS
- Personalización total via `tailwind.config`
- Theming dinámico con `data-theme`
- Bundle size pequeño (solo CSS)

SignalR Client

Elección: `@microsoft/signalr` para comunicación en tiempo real.

- Integración nativa con backend .NET
- Transporte automático (WebSocket > SSE > Long-polling)
- Reconexión automática con backoff exponencial
- Authentication integrada via `accessTokenFactory`

2.5.3. Testing

Backend Tests (xUnit + Moq)

```
cd TriviUpBackend/TriviUpTest  
dotnet test --configuration Release
```

Cobertura actual: ~78% líneas (en este porcentaje se incluyen clases que solo contienen datos y son no testeables, como modelos y DTOs).

Testeado:

- Services (Auth, JWT, Quiz, Game, TurnManager)
- Repositories (User, Quiz, GameHistory)

Para más información, se deben consultar los documentos MANUAL_TÉCNICO.pdf y MANUAL_USUARIO.pdf

3 RESULTADOS Y DISCUSIÓN

3.1. Objetivos Conseguidos

Objetivo	Evidencia
Sistema de juego en tiempo real	SignalR Hub implementado, 15 tests E2E pasando
Creación de cuestionarios	CRUD completo con imágenes
Autenticación JWT + Google OAuth	Flujo completo implementado
Deployment en Railway	https://triviup.up.railway.app
Tests automatizados	Unit tests backend + E2E con Playwright

3.2. Dificultades Encontradas

Dificultad 1: PUT requests bloqueados por nginx/Railway

Problema: Las peticiones PUT a `/users/{id}` retornaban 400 antes de llegar al backend.

Solución temporal: Usar POST para las mismas operaciones (ban/activate funcionan con POST).

Estado: Pendiente de investigación de la configuración de Railway edge.

Dificultad 2: SPA Routing con nginx

Problema: Angular SPA requiere client-side navigation, pero refresh en rutas no-RAÍZ retornaba 404.

Solución:

```
location / {  
  try_files $uri $uri/ /index.html;  
}
```

Dificultad 3: SignalR WebSocket en Railway

Problema: Railway requiere configuración especial para WebSocket upgrades.

Solución:

```
proxy_http_version 1.1;  
proxy_set_header Upgrade $http_upgrade;  
proxy_set_header Connection "upgrade";
```

3.3. Métricas de Calidad

Métrica	Valor
Cobertura backend	~78%
Tests E2E	15 pasando
Tiempo de ejecución E2E	~16s

4 TRABAJO FUTURO (OPCIONAL)

4.1. Mejoras Planificadas

Mejora	Prioridad	Descripción
Fix PUT /users/{id}	Alta	Investigar y resolver bloqueo de Railway
Tests unitarios frontend	Media	Completar cobertura con Vitest
Sistema de logros	Baja	Badges por rachas, quizzes completados
Modo torneo	Baja	Competencia entre múltiples jugadores

4.2. Posibles Ampliaciones

Aplicación móvil: React Native o Flutter

Notifications push: Web Push API para notificar cuando alguien se une a tu sala

Modo offline: Service Worker para jugar sin conexión

Analytics avanzado: Google Analytics o Mixpanel para tracking de usuarios

5 CONCLUSIONES

TriviUp cumple con el objetivo de proporcionar una plataforma de trivia multiplayer en tiempo real accesible desde el navegador. Las decisiones técnicas tomadas (Angular + .NET + SignalR + PostgreSQL) demuestran ser apropiadas para el caso de uso:

1. **Angular 21** proporciona un framework robusto para SPAs con TypeScript obligatorio, reduciendo errores en tiempo de desarrollo.
2. **SignalR** simplifica la comunicación en tiempo real, abstrayendo la complejidad de WebSockets y manejando automáticamente reconexiones.
3. **Clean Architecture** permite separar responsabilidades y facilita el testing de componentes individuales.
4. **PostgreSQL** ofrece una base de datos relacional madura con soporte para JSONB si se necesitara flexibilidad futura.

El proyecto está en un estado funcional con tests automatizados que validan los flujos principales. La deuda técnica principal es la investigación del bloqueo de PUT requests en Railway.

6 BIBLIOGRAFÍA

Tecnologías

- **Angular:** <https://angular.io/>
- **.NET 9:** <https://dotnet.microsoft.com/>
- **PostgreSQL:** <https://www.postgresql.org/>
- **SignalR:** <https://docs.microsoft.com/aspnet/core/signalr/>
- **TailwindCSS:** <https://tailwindcss.com/>
- **DaisyUI:** <https://daisyui.com/>
- **Bruno:** <https://www.usebruno.com/>
- **Playwright:** <https://playwright.dev/>

Herramientas

- **Railway:** <https://railway.app/>
- **Visual Studio Code:** <https://code.visualstudio.com/>
- **Rider:** <https://www.jetbrains.com/rider/>
- **WebStorm:** <https://www.jetbrains.com/webstorm/>
- **Git:** <https://git-scm.com/>

ANEXOS

Anexo 1: Estimación de costes

Requisitos funcionales		
RF	Nombre	Horas estimadas
RF-001	Registro de usuario con nombre, email y contraseña	6
RF-002	Inicio de sesión con email y contraseña	4
RF-003	Inicio de sesión con Google OAuth	8
RF-004	Edición de perfil (nombre, email, contraseña)	4
RF-005	Subida de foto de perfil (máx 5MB, JPG/PNG/GIF/WebP)	6
RF-006	Crear cuestionario con nombre, visibilidad y preguntas	10
RF-007	Editar cuestionario propio	6
RF-008	Eliminar cuestionario propio	4
RF-009	Listar mis cuestionarios	4
RF-010	Explorar cuestionarios públicos con búsqueda y paginación	8
RF-011	Ver detalle de un cuestionario	4
RF-012	Dar like a cuestionario público	2
RF-013	Quitar like a cuestionario público	2
RF-014	Crear sala de juego con código único (6 caracteres)	8
RF-015	Unirse a sala (como usuario o como invitado)	10
RF-016	Iniciar juego (mínimo 2 jugadores, solo owner)	6
RF-017	Responder pregunta en turno (21 segundos)	6
RF-018	Sistema de turnos rotativos	8
RF-019	Pausar juego (solo owner)	4
RF-020	Reanudar juego (solo owner)	4
RF-021	Expulsar jugador (solo owner)	4
RF-022	Ver resultados finales y clasificación	8
RF-023	Ver historial de partidas	6
RF-024	Dashboard con estadísticas del sistema	12
RF-025	Listar todos los usuarios	4
RF-026	Editar datos de cualquier usuario	6
RF-027	Banear usuario	4
RF-028	Activar usuario baneado	4
		162

Requisitos no funcionales ▾		
RNF ▾	Nombre ▾	Horas estimadas ▾
RNF-001	Tiempo de respuesta APIs < 500ms (p95)	8
RNF-002	Latencia SignalR < 100ms	12
RNF-003	Carga de página < 3 segundos	10
RNF-004	JWT con expiración 60 minutos	4
RNF-005	Contraseñas hasheadas con bcrypt	4
RNF-006	HTTPS obligatorio en producción	2
RNF-007	Validación de todos los inputs	8
RNF-008	Autorización por roles (USER/ADMIN)	6
RNF-009	Uptime > 99%	10
RNF-010	Health check endpoint	2
RNF-011	Soporte para usuarios concurrentes	12
RNF-012	Preparado para múltiples instancias	8
RNF-013	Diseño responsive (móvil, tablet, desktop)	15
RNF-014	Compatible con Chrome, Firefox, Safari, Edge	10
RNF-015	Code coverage backend > 80%	12
RNF-016	Frontend testeado mediante Playwright	12
		135

Requisitos de información ▾		
RI ▾	Nombre ▾	Horas estimadas ▾
RI1	Modelo + DTOs de usuario	3
RI2	Modelo + DTOs de cuestionario	3
RI3	Modelo + DTOs de pregunta	1
RI4	Modelo + DTOs de respuesta	1
RI5	Modelo + DTOs de partida	4
		12

Reglas de negocio		
RN	Nombre	Horas estimadas
RN-001	Username único en el sistema	2
RN-002	Email único en el sistema	2
RN-003	Usuario baneado no puede jugar ni iniciar sesión	2
RN-004	Código de juego generado por el sistema (único)	2
RN-005	Mínimo 1 pregunta por cuestionario	2
RN-006	Cada pregunta tiene 2-4 respuestas, exactamente 1 correcta	2
RN-007	Solo el owner puede iniciar el juego	2
RN-008	Mínimo 2 jugadores para iniciar	2
RN-009	Máximo 10 jugadores por sala	2
RN-010	Tiempo máximo de respuesta: 21 segundos	2
RN-011	Puntuación: $100 + (\text{tiempo_restante} \times 10)$, máximo 300	2
		22
HORAS DE TRABAJO TOTALES		
		331
PRECIO POR HORA DE TRABAJO		
		135,00 €
PRECIO POR HORA EXTRA DE TRABAJO		
		160,00 €
COSTE TOTAL DE LAS HORAS DE TRABAJO		
		44.685,00 €

ESTIMACIÓN DE COSTES				
Nº	Concepto	Desglose		Coste
		Precio por hora	Horas	
1	Horas de trabajo estimadas por requisitos	135,00 €	331	44.685,00 €
		Precio por hora extra	15% de las horas totales	
2	Horas extra para cubrir imprevistos	160,00 €	49,65	7.944,00 €
		Costes antes de margen	17% sobre los costes	
3	Margen de beneficios	52.629,00 €	0,17	8.946,93 €
				61.575,93 €

Anexo 2: Manual de Usuario

Ver archivo [MANUAL_USUARIO.pdf](#) para instrucciones detalladas de uso.

Anexo 3: Documentación Técnica Adicional

Ver archivo [MANUAL_TECNICO.pdf](#) para prompts de generación de diagramas con IA.

Anexo 4: seed Data

Al iniciar en desarrollo o si `ASPNETCORE_ENVIRONMENT=Development`:

- **Users:** admin (ADMIN), user (USER), testuser (USER)
- **Quizzes:** 6 cuestionarios (5 públicos, 1 privado) con preguntas de ejemplo

Anexo 5: API Reference

Autenticación

Endpoint	Método	Auth	Descripción
<code>/auth/signup</code>	POST	No	Registro
<code>/auth/signin</code>	POST	No	Login
<code>/auth/google</code>	GET	No	OAuth Google
<code>/auth/google/callback</code>	GET	No	OAuth Callback

Usuarios

Endpoint	Método	Auth	Descripción
<code>/users/me</code>	GET	JWT	Usuario actual
<code>/users/me</code>	PUT	JWT	Actualizar perfil
<code>/users/profile-photo</code>	PUT	JWT	Subir foto
<code>/users/profile-photo</code>	DELETE	JWT	Eliminar foto
<code>/users</code>	GET	ADMIN	Listar usuarios
<code>/users/{id}</code>	PUT	ADMIN	Actualizar usuario
<code>/users/{id}/ban</code>	POST	ADMIN	Banear
<code>/users/{id}/activate</code>	POST	ADMIN	Activar

Cuestionarios

Endpoint	Método	Auth	Descripción
/api/cuestionarios	POST	JWT	Crear
/api/cuestionarios	GET	JWT	Listar (pág.)
/api/cuestionarios/{id}	GET	JWT	Detalle
/api/cuestionarios/{id}	PUT	JWT (owner)	Actualizar
/api/cuestionarios/{id}	DELETE	JWT (owner)	Eliminar
/api/cuestionarios/mis-cuestionarios	GET	JWT	Mis cuestionarios
/api/cuestionarios/gamecode/{code}	GET	JWT	Por código

Quizzes Públicos

Endpoint	Método	Auth	Descripción
/api/quizzes/public	GET	No	Listar públicos
/api/quizzes/{id}/like	POST	No	Dar like
/api/quizzes/{id}/like	DELETE	No	Quitar like
/api/quizzes/{id}/visit	POST	No	Registrar visita

Juego

Endpoint	Método	Auth	Descripción
/api/game/history	GET	JWT	Historial
/api/game/{id}	GET	JWT	Detalle partida
/hubs/game	WebSocket	JWT/anon	Juego en tiempo real

Admin

Endpoint	Método	Auth	Descripción
/api/admin/stats	GET	ADMIN	Estadísticas

Anexo 6: Enlaces

Repositorio Backend:

<https://github.com/charlieecy/TriviUpBackend>

Repositorio Frontend:

<https://github.com/Sggz221/TriviUpFrontend>

Aplicación desplegada:

<https://triviup.up.railway.app/>